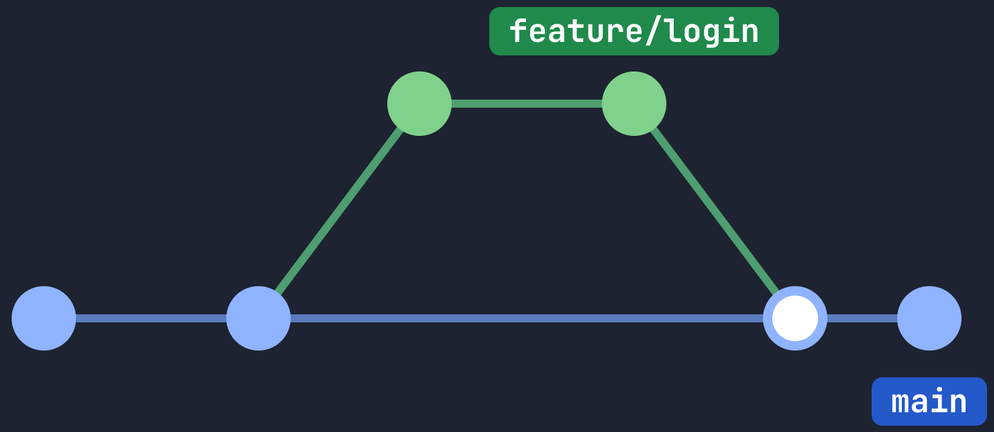
KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

Git 브랜치, 눈으로 보기

따로 작업하고, 합치고,
충돌을 푸는 법



오늘의 순서

1 브랜치란?
점, 이름표, HEAD
3–6쪽

2 만들고 옮기기
branch · checkout · switch
7–14쪽

3 합치기
merge · 이름 규칙
15–19쪽

4 갈라진 길
merge 커밋
20–25쪽

5 되돌리기
git reset 옵션 세 가지
26–38쪽

6 충돌 풀기
표시 읽고 고치기
39–45쪽

7 GitHub에서 함께
push · Pull Request
46–56쪽

8 충돌 시나리오
다섯 가지 상황
57–68쪽

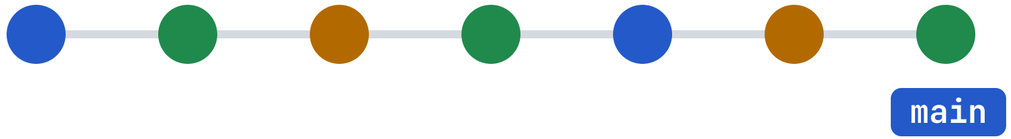
9 막힐 때
자주 나는 에러
69–71쪽

마지막 72쪽은 한 장 요약이에요.

브랜치란?

왜 브랜치를 쓸까요?

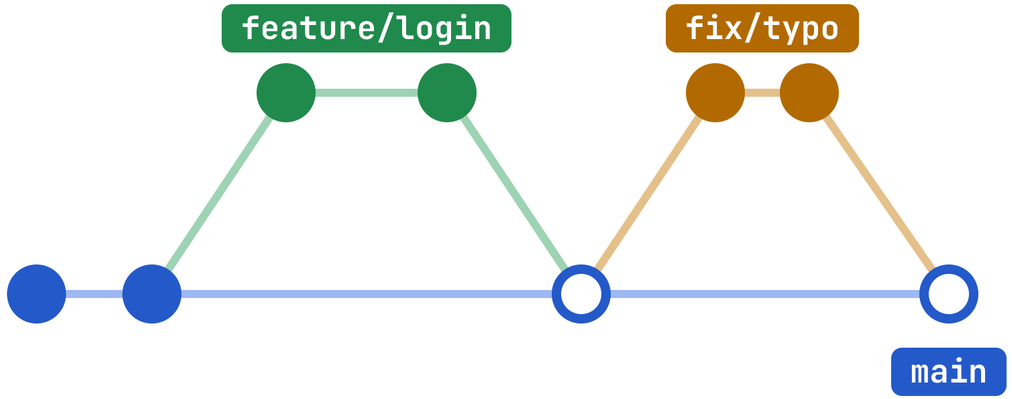
⚠ 모두 main에 바로 커밋



반쯤 만든 기능이 main에 섞여요

누구 작업이 문제인지 찾기 어려워요

✓ 각자 브랜치에서 작업



다 만든 다음에 main으로 합쳐요

main은 늘 잘 돌아가는 상태로 남아요

브랜치 = 본 줄기(main)는 그대로 두고, 옆으로 갈래를 내서 작업하는 방법

점 하나가 커밋 하나예요

```
$ git log --oneline
d6b044a (HEAD → main, origin/main) docs: README 추가
5ec913c feat: 인사말 추가
9e6cce4 feat: 인사 파일 추가
```



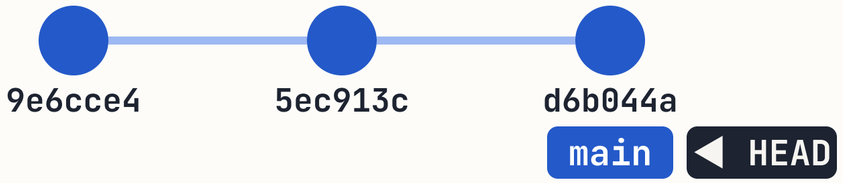
왼쪽이 먼저 한 커밋, 오른쪽이 최근 커밋이에요.

main은 마지막 커밋에 붙은 이름표예요



새 커밋을 하면 이름표가 새 커밋으로 옮겨 가요.

HEAD는 지금 내가 있는 곳이에요

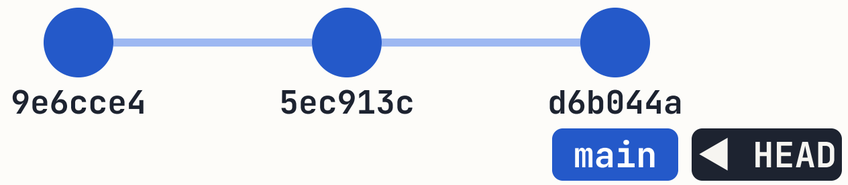


커밋하면 HEAD가 붙은 이름표가 앞으로 가요.

만들고 옮기기

브랜치 목록 보기

```
$ git branch
* main
```

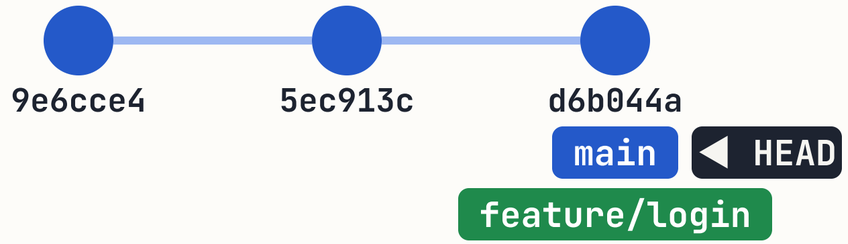


* 표시가 지금 있는 브랜치예요. 그래프의 HEAD와 같은 뜻이에요.

만들고 옮기기

브랜치 만들기: 이름표가 하나 더 생겨요

```
$ git branch feature/login
```

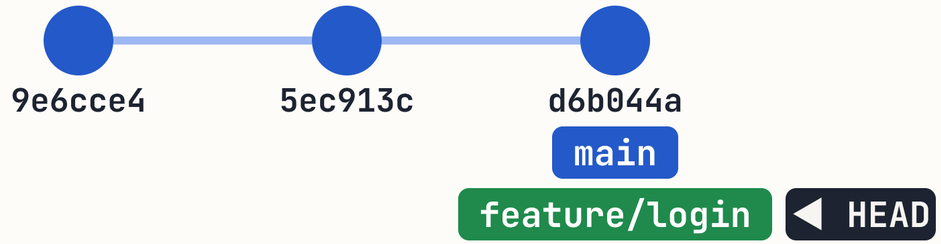


커밋은 복사되지 않아요. HEAD는 아직 main에 있어요.

만들고 옮기기

브랜치로 옮겨 가기: HEAD가 움직여요

```
$ git checkout feature/login
Switched to branch 'feature/login'
```

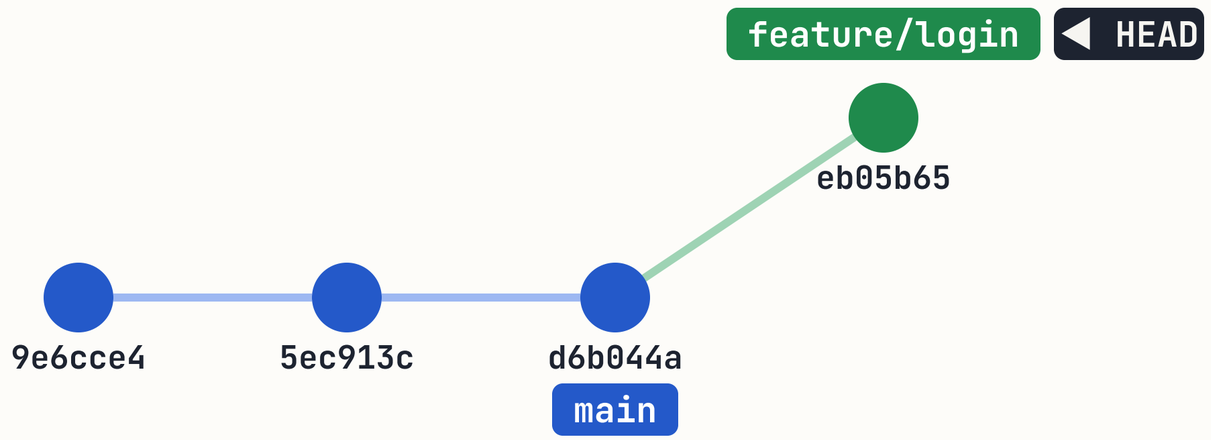


이제부터 하는 커밋은 feature/login에 쌓여요.

만들고 옮기기

커밋하면 feature/login만 앞으로 가요

```
$ git add login.md
$ git commit -m "feat: 로그인 화면 추가"
[feature/login eb05b65] feat: 로그인 화면 추가
```

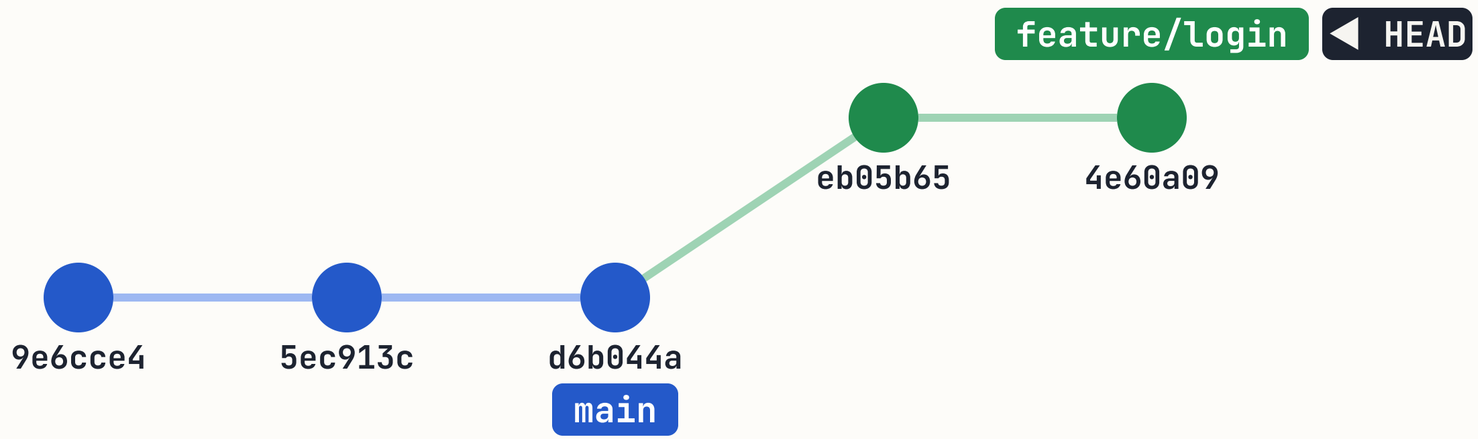


main은 제자리예요.

만들고 옮기기

한 번 더 커밋하면 한 칸 더

```
$ git add login.md
$ git commit -m "feat: 비밀번호 입력 추가"
[feature/login 4e60a09] feat: 비밀번호 입력 추가
```

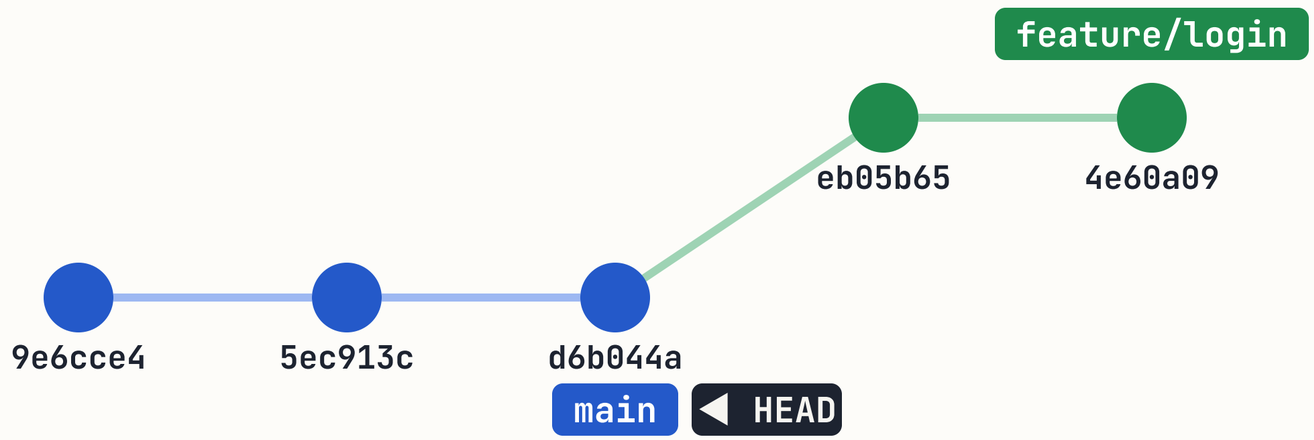


기능이 끝날 때까지 브랜치에서 여러 번 커밋해도 돼요.

만들고 옮기기

main으로 돌아가기

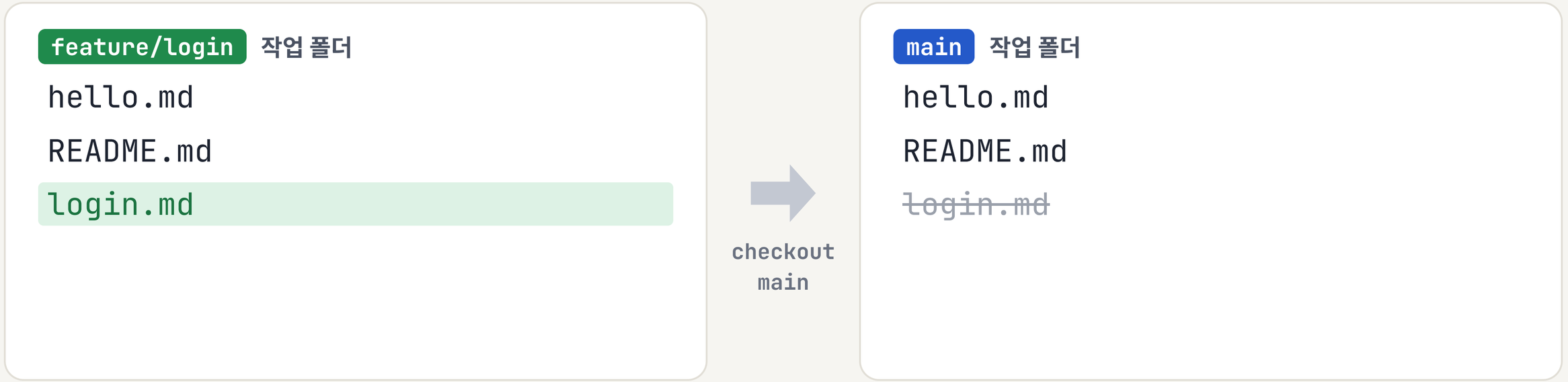
```
$ git checkout main
Switched to branch 'main'
```



HEAD만 main으로 돌아왔어요.

만들고 옮기기

브랜치를 옮기면 폴더 파일도 바뀌어요



login.md는 사라진 게 아니에요. feature/login으로 가면 다시 보여요.

만들고 옮기기

브랜치만 옮기는 명령, git switch

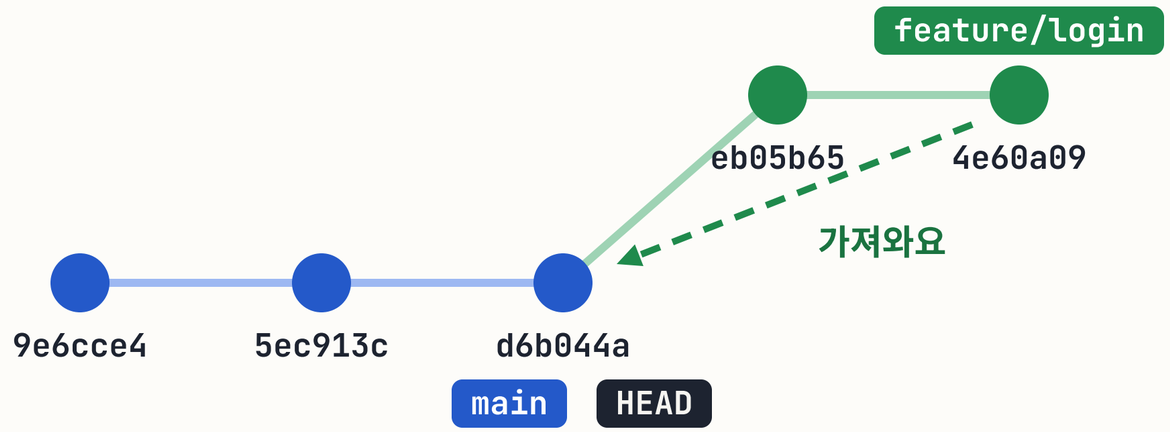
	git checkout	git switch
브랜치로 옮겨 가기	git checkout 이름	git switch 이름
만들고 바로 옮겨 가기	git checkout -b 이름	git switch -c 이름

```
$ git switch feature/login
Switched to branch 'feature/login'
$ git switch -c feature/login-button
Switched to a new branch 'feature/login-button'
```

하는 일과 출력은 같아요. 이 덱은 checkout으로 보여 주고, switch로 쳐도 돼요.

합치기

merge는 받는 쪽에서 해요



받는 쪽

먼저 checkout으로 가 있는 브랜치

가져올 쪽

git merge 뒤에 적는 이름

1 **받을 브랜치로 이동**

```
git checkout main
```

2 **가져올 브랜치 이름을 적어요**

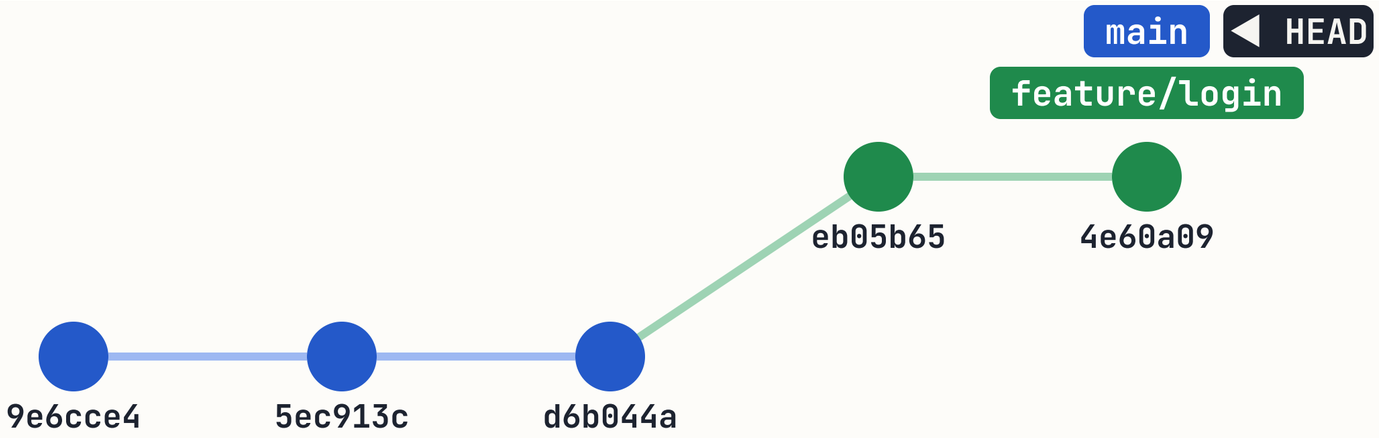
```
git merge feature/login
```

02에서 본 git pull origin main도 받아 온 커밋을 이 방식으로 merge해요.

합치기

Fast-forward: main 이름표만 앞으로

```
$ git merge feature/login
Updating d6b044a..4e60a09
Fast-forward
```

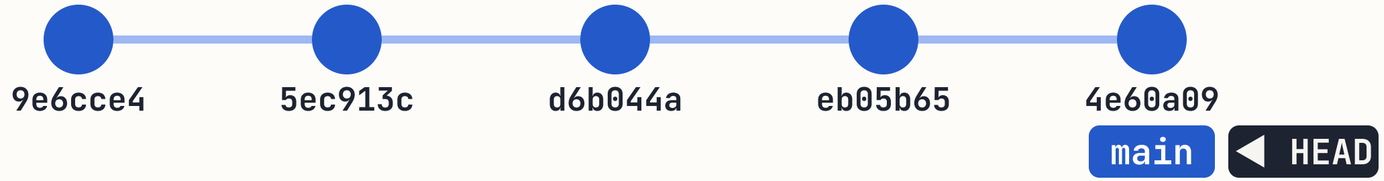


main 쪽에 새 커밋이 없으면 이름표만 당겨요.

합치기

다 쓴 브랜치 지우기

```
$ git branch -d feature/login
Deleted branch feature/login (was 4e60a09).
```



이름표만 떼요. 커밋은 main에 한 줄로 남아요.

합치기 · 브랜치 이름

브랜치 이름은 타입/하는 일

feature/login-button

타입 · 슬래시 · 하는 일(소문자, 하이픈)

feature/
새 기능
feature/login-button

fix/
버그 수정
fix/login-error

docs/
문서
docs/login-guide

refactor/
구조 개선
refactor/user-api

chore/
설정, 패키지
chore/eslint-setup

새 기능은 feature/예요. 커밋 메시지 타입 feat:와 헷갈리지 않게 봐요.

합치기 · 브랜치 이름

이렇게 짓고, 이건 피해요

이렇게

`feature/login-button`

`fix/12-login-error` (이슈 12번)

이건 피해요

`login`, `내작업`, `final2` (무슨 일인지 모름)

`Feature/Login Button` (대문자, 띄어쓰기)

소문자와 하이픈(-)만 쓰고, 무슨 일인지 이름에 적어요.

갈라진 길 합치기

새 브랜치 docs/login-guide 만들고 옮겨 가기

```
$ git checkout -b docs/login-guide
Switched to a new branch 'docs/login-guide'
```

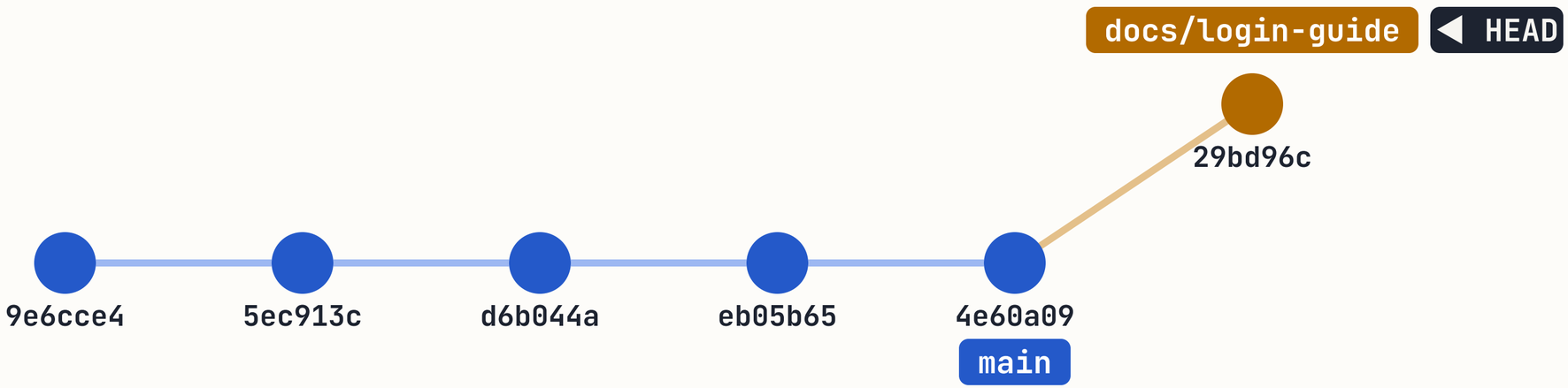


-b는 만들기와 옮기기를 한 번에 해요.

갈라진 길 합치기

docs/login-guide에서 커밋

```
$ git add README.md
$ git commit -m "docs: README에 로그인 안내 추가"
[docs/login-guide 29bd96c] docs: README에 로그인 안내 추가
```

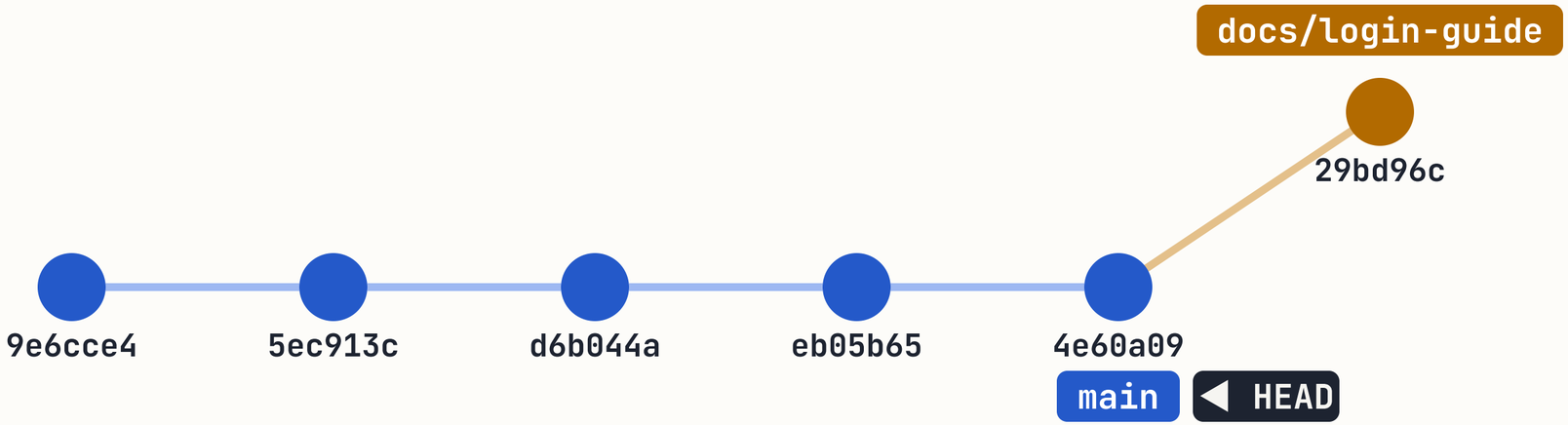


docs/login-guide만 한 칸 앞으로 가요.

갈라진 길 합치기

다시 main으로

```
$ git checkout main
Switched to branch 'main'
```

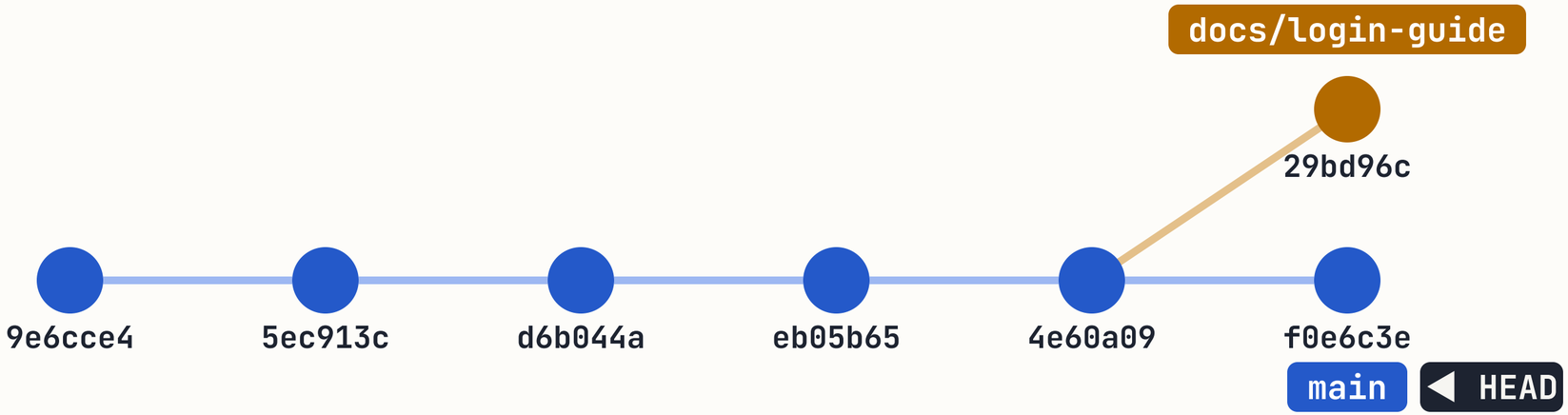


이번엔 main에서도 커밋해 볼게요.

갈라진 길 합치기

main에도 새 커밋: 길이 두 갈래가 됐어요

```
$ git add hello.md
$ git commit -m "feat: 반가움 인사 추가"
[main f0e6c3e] feat: 반가움 인사 추가
```

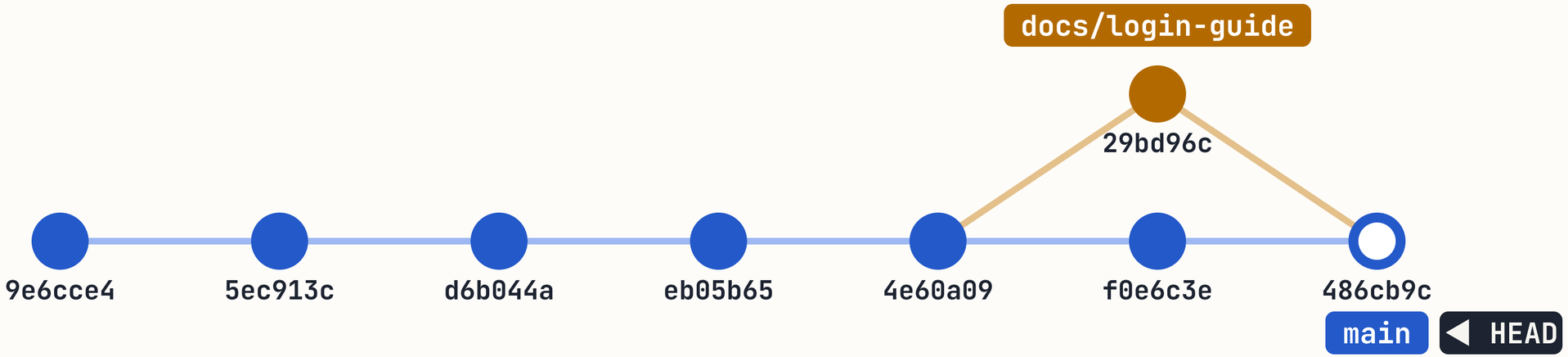


이번엔 이름표만 당겨서는 합칠 수 없어요.

갈라진 길 합치기

merge 커밋이 두 길을 이어요

```
$ git merge docs/login-guide
Merge made by the 'ort' strategy.
```

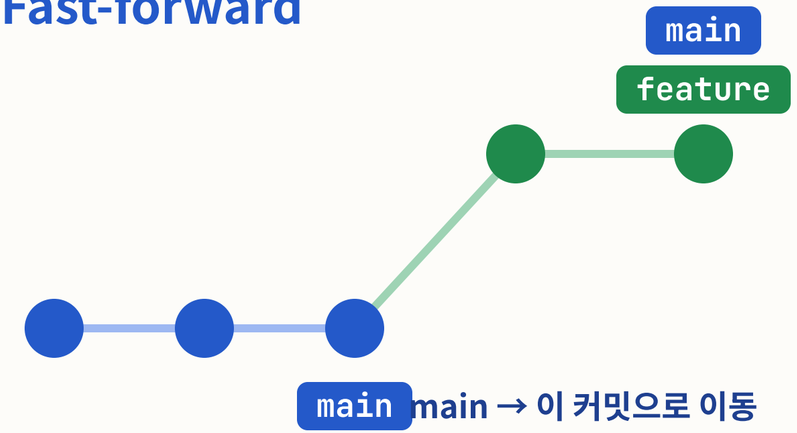


흰 점이 새로 생긴 merge 커밋이에요. 메시지 편집기가 열리면 그대로 저장하고 닫아요.

개념 정리

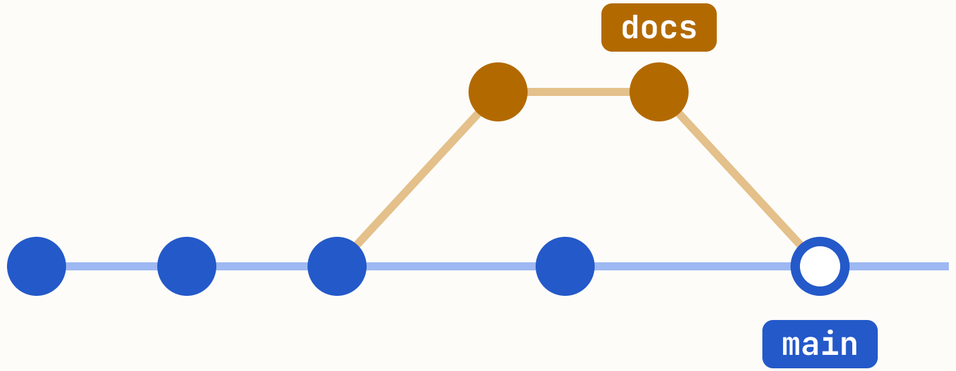
merge는 두 가지 모습으로 끝나요

Fast-forward



main에 새 커밋이 없을 때
이름표만 앞으로 당겨요

merge 커밋



양쪽에 새 커밋이 있을 때
둘을 잇는 커밋을 새로 만들어요

명령은 똑같이 git merge 브랜치이름. 어느 쪽이 될지는 Git이 정해요.

되돌리기

reset은 이름표를 뒤로 옮겨요

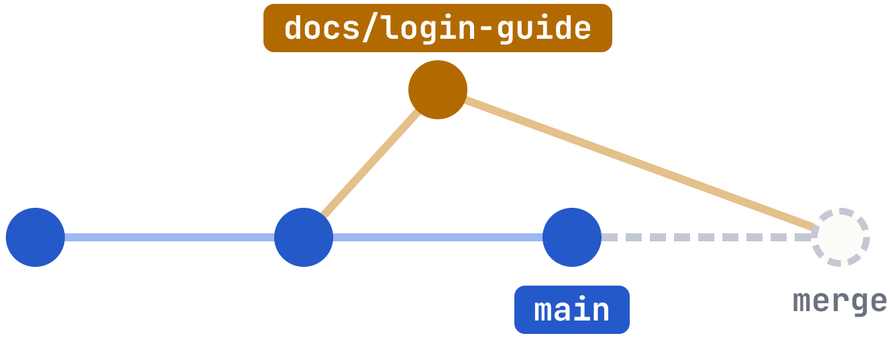
케이스 1 · 일반 커밋

실수한 커밋 하나를 빼요



케이스 2 · merge 커밋

합친 것을 없던 일로 해요

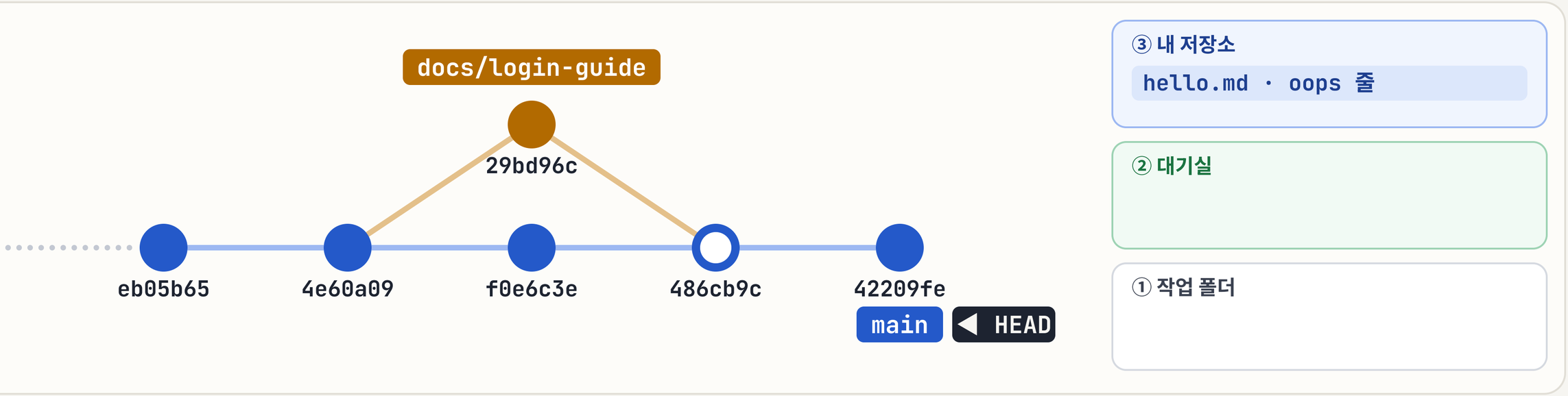


`git reset 옵션 HEAD~1` 지금 브랜치 이름표를 한 칸 아래 커밋으로 옮겨요

되돌리기 · 케이스 1 일반 커밋

실수로 커밋했어요, 아직 push 전이에요

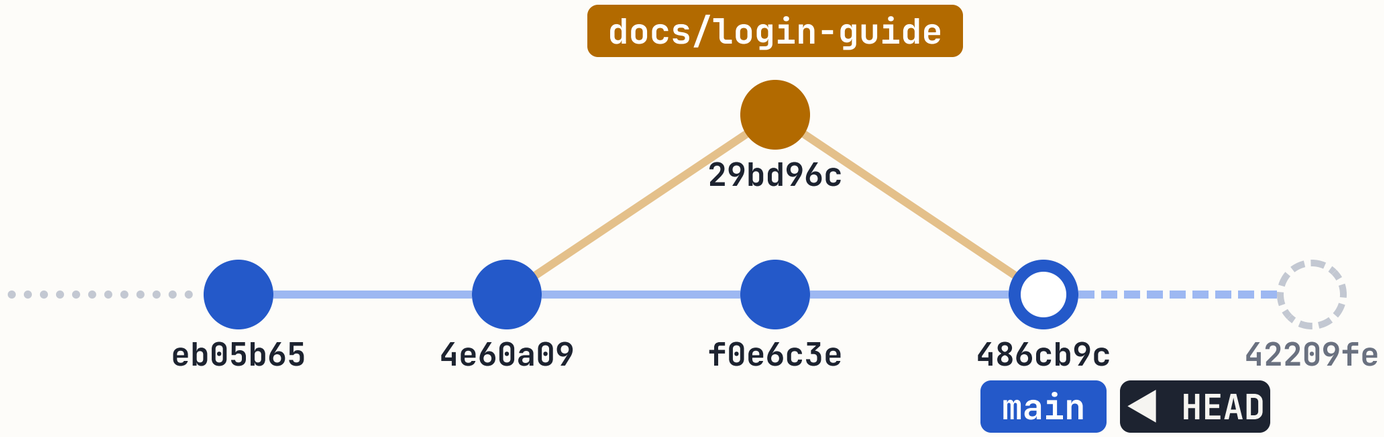
```
$ git add hello.md
$ git commit -m "feat: 실수한 커밋"
[main 42209fe] feat: 실수한 커밋
```



이 커밋 하나를 없던 일로 해 볼게요.

--soft: 변경이 대기실에 남아요

```
$ git reset --soft HEAD~1
$ git status
Changes to be committed:
    modified: hello.md
```



③ 내 저장소

② 대기실

`hello.md · oops 줄`

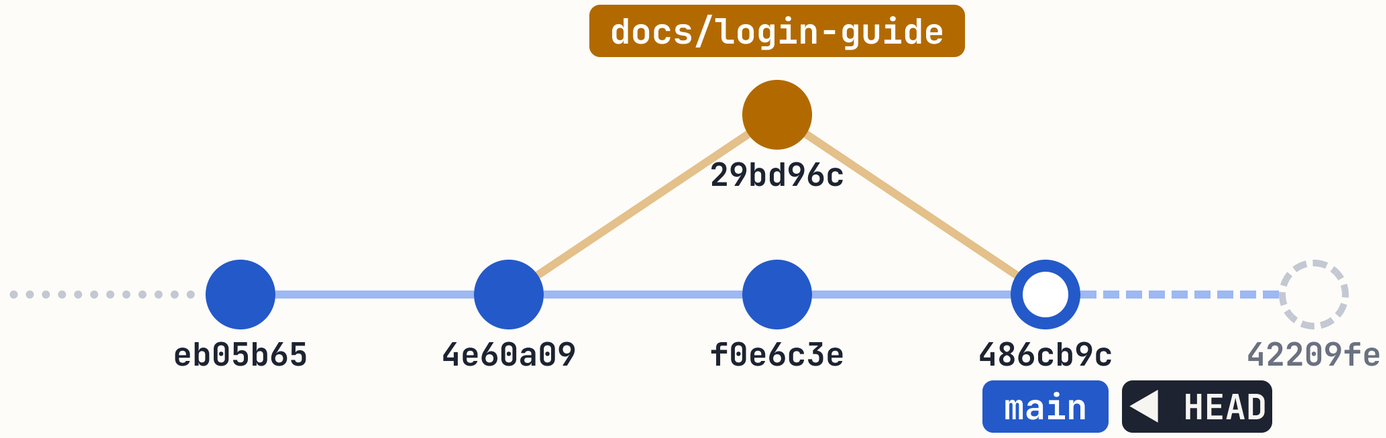
① 작업 폴더

main이 한 칸 뒤로 가고, oops 줄은 add된 채로 남아요.

되돌리기 · 케이스 1 일반 커밋

--mixed (기본값): 변경이 작업 폴더에 남아요

```
$ git reset HEAD~1
Unstaged changes after reset:
M    hello.md
```



③ 내 저장소

② 대기실

① 작업 폴더

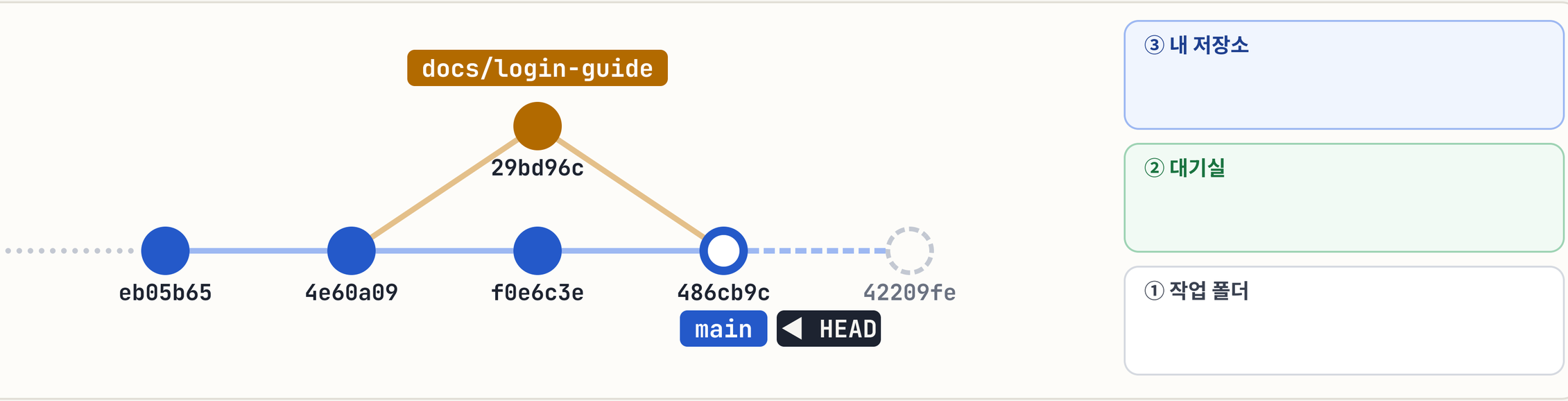
`hello.md · oops 줄`

옵션을 안 쓰면 --mixed예요. oops 줄은 파일에 그대로 있어요.

되돌리기 · 케이스 1 일반 커밋

--hard: 변경까지 지워요

```
$ git reset --hard HEAD~1
HEAD is now at 486cb9c Merge branch 'docs/login-guide'
$ git status
nothing to commit, working tree clean
```

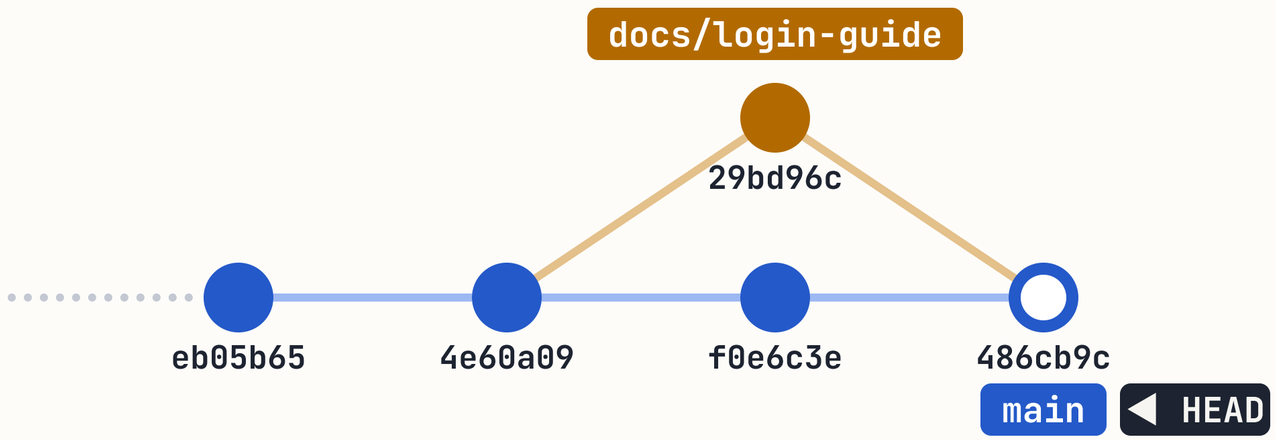


oops 줄이 파일에서도 사라졌어요. 이 변경은 다른 곳에 남아 있지 않아요.

되돌리기 · 케이스 2 merge 커밋

이번엔 merge 커밋을 되돌려요

```
$ git log --oneline
486cb9c (HEAD → main) Merge branch 'docs/login-guide'
f0e6c3e feat: 반가움 인사 추가
29bd96c (docs/login-guide) docs: README에 로그인 안내 추가
```



③ 내 저장소

README.md · 로그인 안내

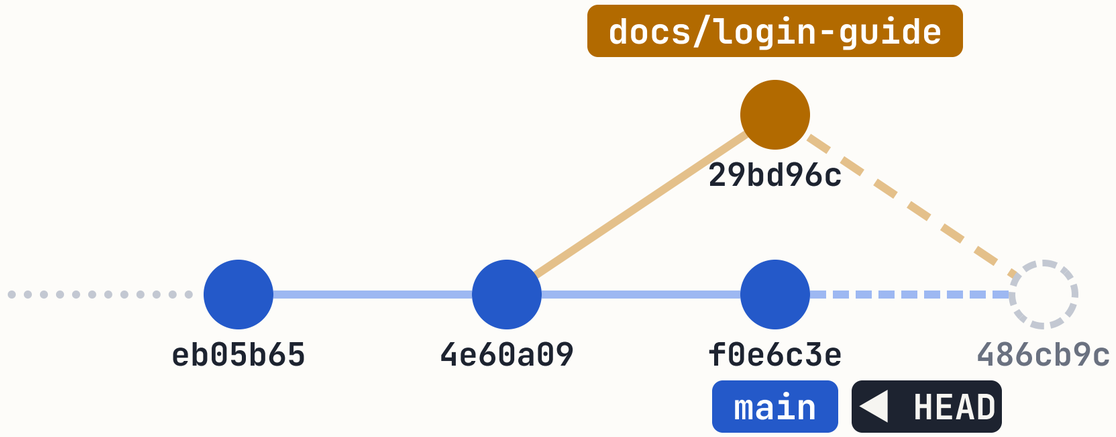
② 대기실

① 작업 폴더

merge 커밋도 같은 명령으로 한 칸 뒤로 가요.

--soft: 변경이 대기실에 남아요

```
$ git reset --soft HEAD~1
$ git status
Changes to be committed:
    modified: README.md
```



③ 내 저장소

② 대기실

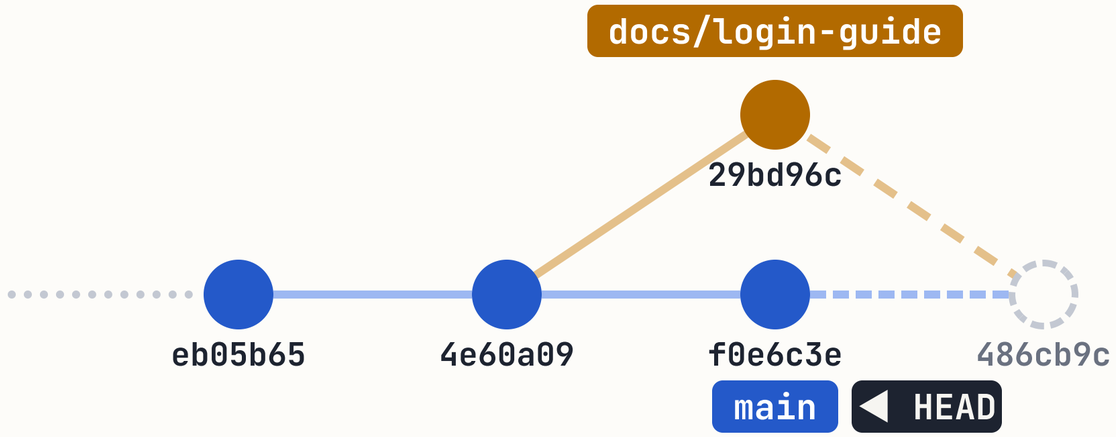
README.md · 로그인 안내

① 작업 폴더

main은 merge 전 커밋으로 돌아가고, 합쳐 온 변경은 대기실에 있어요.

--mixed (기본값): 변경이 작업 폴더에 남아요

```
$ git reset HEAD~1
Unstaged changes after reset:
M    README.md
```



③ 내 저장소

② 대기실

① 작업 폴더

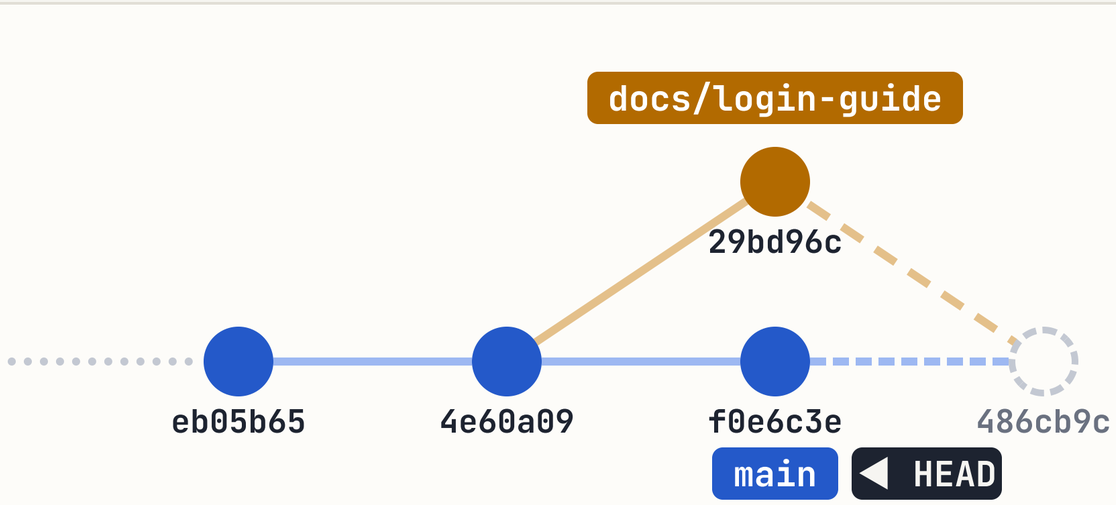
README.md · 로그인 안내

옵션을 안 쓰면 --mixed예요. 고친 내용은 파일에 그대로 있어요.

되돌리기 · 케이스 2 merge 커밋

--hard: 변경까지 지워요

```
$ git reset --hard HEAD~1
HEAD is now at f0e6c3e feat: 반가움 인사 추가
$ git status
nothing to commit, working tree clean
```



③ 내 저장소

② 대기실

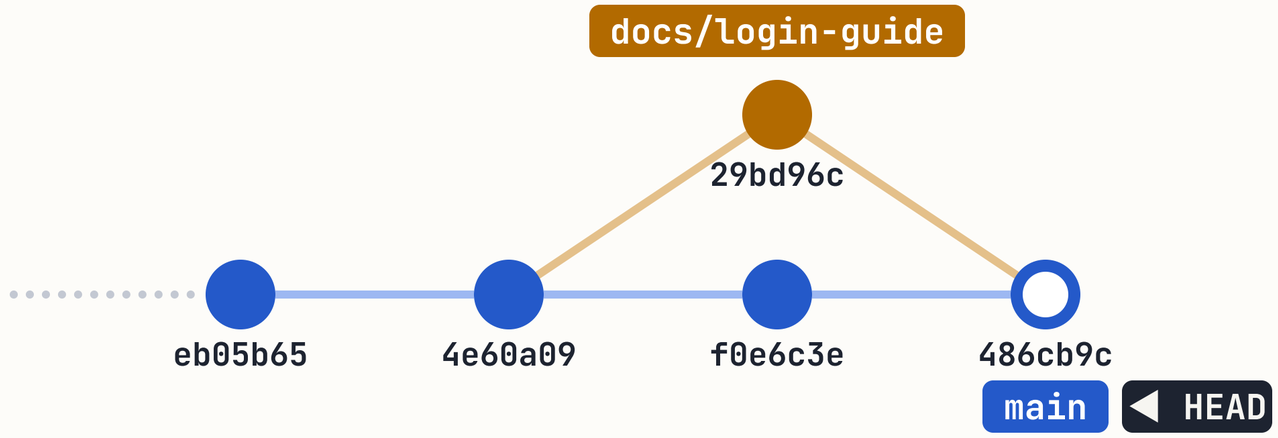
① 작업 폴더

--hard는 커밋하지 않은 변경까지 지워요. 쓰기 전에 git status가 깨끗한지 봐요.

되돌리기 · 케이스 2 merge 커밋

브랜치가 남아 있으면 다시 합칠 수 있어요

```
$ git merge docs/login-guide
Merge made by the 'ort' strategy.
```



③ 내 저장소

README.md · 로그인 안내

② 대기실

① 작업 폴더

docs/login-guide를 지우기 전이라 merge를 몇 번이든 다시 할 수 있어요.

되돌리기 · 정리

세 옵션, 무엇이 어디에 남을까?

	③ 내 저장소	② 대기실	① 작업 폴더	merge한 브랜치
`--soft`	커밋 취소	변경이 여기 남음	그대로	그대로
`--mixed` (기본)	커밋 취소	비움	변경이 여기 남음	그대로
`--hard`	커밋 취소	비움	변경이 사라짐	그대로

--hard로 지운 변경은 일반 커밋이면 남는 곳이 없고, merge면 브랜치에 남아요.

되돌리기 · 정리

언제 무엇으로 되돌릴까?

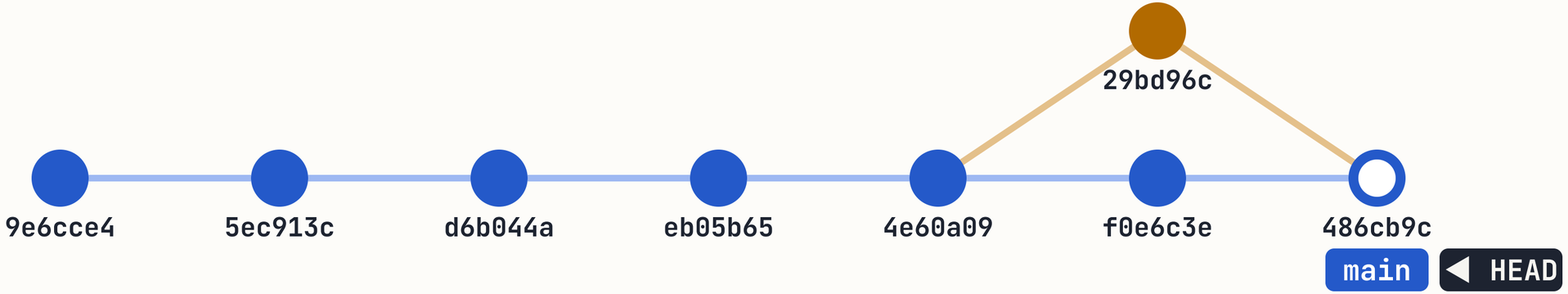
충돌이 나서 merge가 멈췄어요	`git merge --abort` 합치기 전으로 돌아가요
커밋했는데 아직 push 전이에요	`git reset HEAD~1` 고친 내용은 작업 폴더에 남아요
merge를 끝냈고, 아직 push 전이에요	`git reset --hard HEAD~1` git status가 깨끗한지 먼저 봐요
push까지 했어요	`git revert 커밋` merge 커밋이면 팀에 먼저 알려요

기준은 하나예요. push 전이면 reset, push 후면 revert.

되돌리기 · 정리

확인이 끝나면 docs/login-guide 지우기

```
$ git branch -d docs/login-guide
Deleted branch docs/login-guide (was 29bd96c).
```



이름표만 사라지고 커밋은 main 기록 안에 남아요.

충돌 풀기

충돌은 언제 나요?

다른 파일을 고침
알아서 합쳐요

같은 파일, 멀리 떨어진 줄
알아서 합쳐요

같은 줄, 바로 옆 줄
충돌: 사람이 골라요

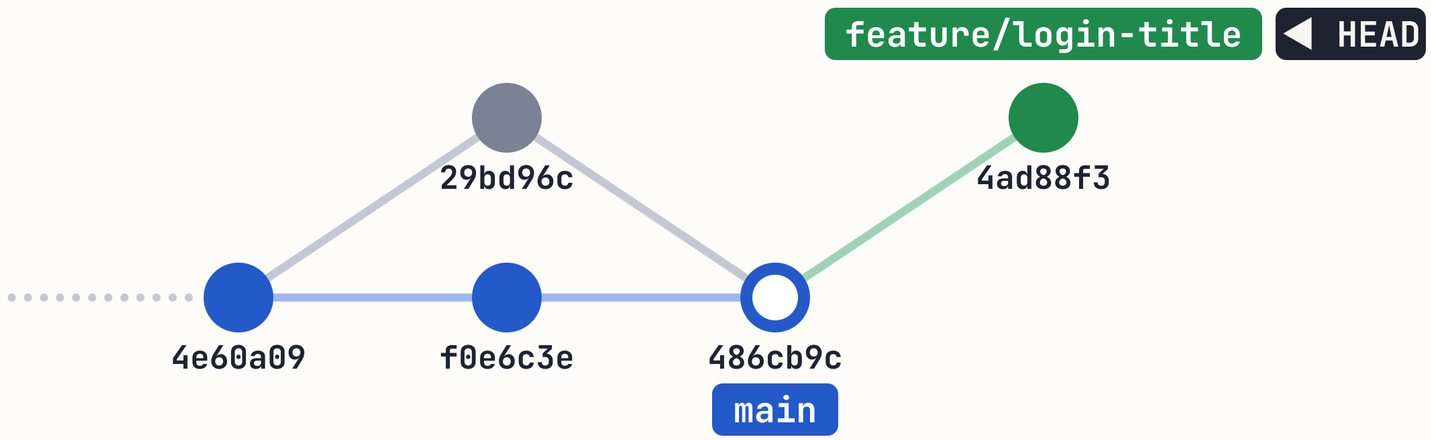
한쪽은 지우고, 한쪽은 고침
충돌: 사람이 골라요

충돌은 Git이 고를 수 없는 곳만 사람에게 물어보는 거예요.

충돌 시나리오 1 · 같은 줄

feature/login-title에서 제목을 바꿔요

```
$ git checkout -b feature/login-title
$ git add login.md
$ git commit -m "feat: 로그인 제목 바꾸기"
[feature/login-title 4ad88f3] feat: 로그인 제목 바꾸기
```



feature/login-title의 login.md

```
# 로그인하기
```

아이디

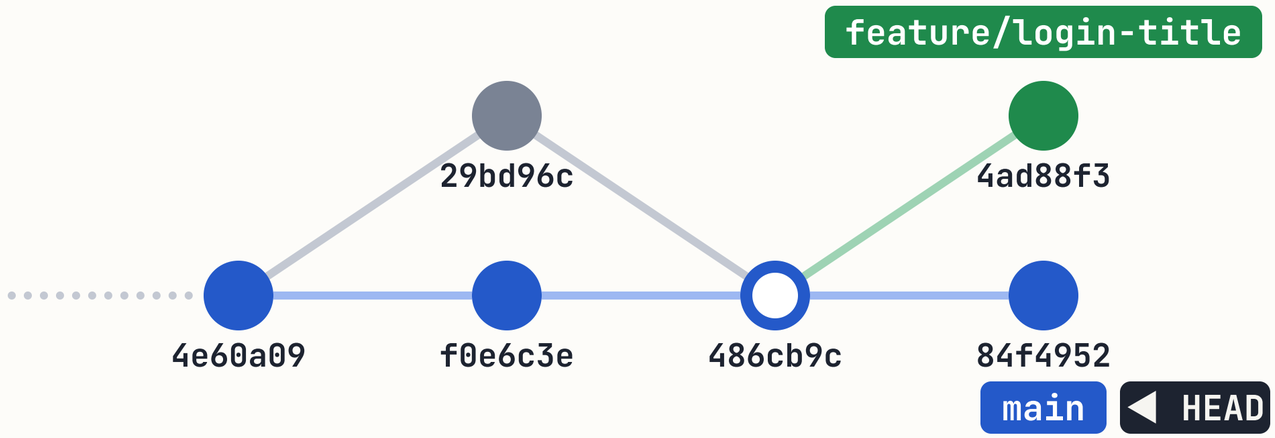
비밀번호

login.md 첫 줄을 # 로그인하기로 바꿨어요.

충돌 시나리오 1 · 같은 줄

main에서는 같은 줄을 다르게

```
$ git checkout main
$ git add login.md
$ git commit -m "feat: 로그인 제목 다듬기"
[main 84f4952] feat: 로그인 제목 다듬기
```



main의 login.md

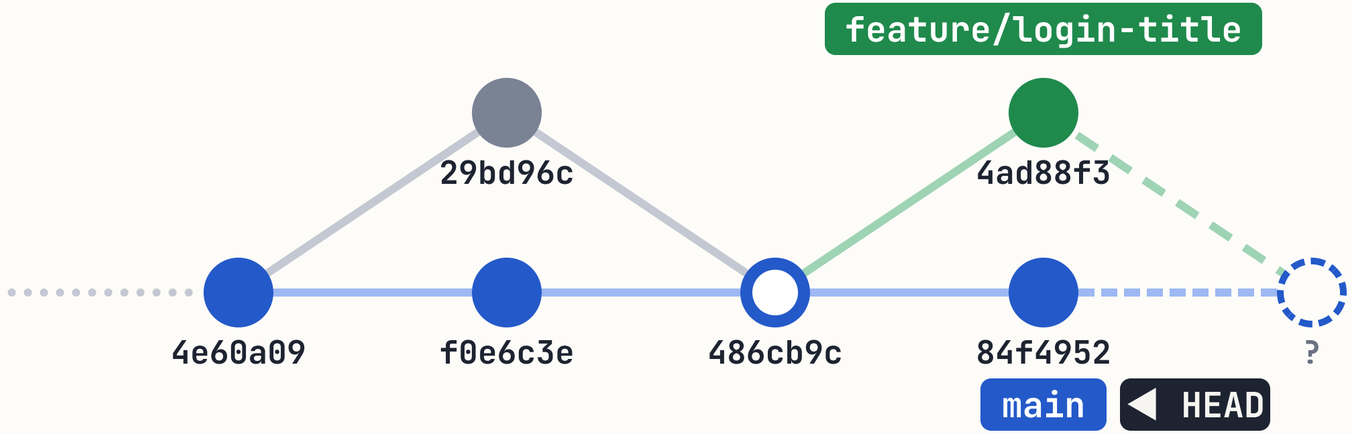
```
# 로그인 페이지
아이디
비밀번호
```

같은 줄을 양쪽에서 다르게 고쳤어요.

충돌 시나리오 1 · 같은 줄

합치려니 멈췄어요: 충돌

```
$ git merge feature/login-title
CONFLICT (content): Merge conflict in login.md
Automatic merge failed; fix conflicts and then commit the result.
```



Git이 두 제목 중 무엇을 남길지 몰라서 나에게 물어보는 거예요.

충돌 풀기

충돌 표시 읽는 법

login.md

```
<<<<<<< HEAD
# 로그인 페이지
=======
# 로그인하기
>>>>>>> feature/login-title
아이디
비밀번호
```

지금 내 브랜치(main) 내용

합치려는 브랜치(feature/login-title) 내용

푸는 순서

1 남길 내용으로 고치기
둘 다 남겨도 돼요

2 표시 세 줄 지우고 저장

3 `git add login.md`

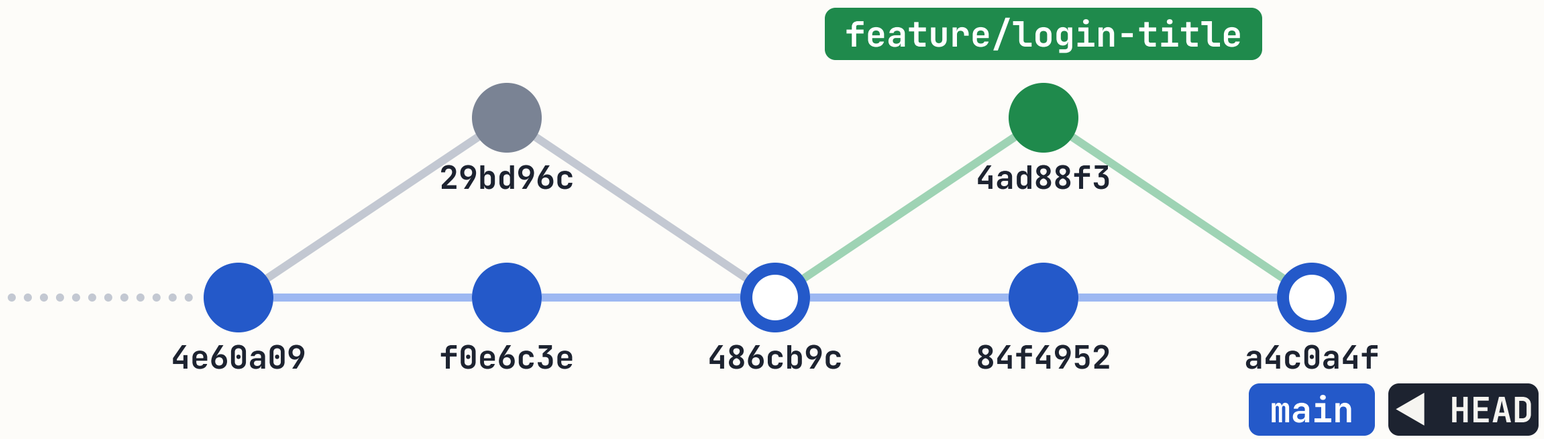
4 `git commit -m "..."`

<<<<<<<, =======, >>>>>>> 줄이 남은 채로 커밋하지 않아요.

충돌 시나리오 1 · 같은 줄

고치고 add, commit하면 끝

```
$ git add login.md
$ git commit -m "fix: 로그인 제목 충돌 해결"
[main a4c0a4f] fix: 로그인 제목 충돌 해결
```



고친 login.md

```
# 로그인하기
아이디
비밀번호
```

이 커밋이 두 길을 잇는 merge 커밋이에요.

충돌 풀기

헷갈리면 merge 전으로 돌아가요

```
$ git merge feature/login-email
CONFLICT (content): Merge conflict in login.md
Automatic merge failed; fix conflicts and then commit the result.
$ git merge --abort
$ git status
On branch main
nothing to commit, working tree clean
```

merge --abort는 합치기 전 모습으로 돌려놔요. 천천히 다시 해도 돼요.

GitHub에서 함께

팀에서는 이 순서로 일해요

내 컴퓨터

GitHub 웹사이트

내 컴퓨터

1



브랜치 만들기

```
git checkout
-b feature/
login-button
```

2



작업하고 커밋

```
add, commit
```

3



브랜치 올리기

```
git push
origin
feature/
login-button
```

4



합쳐 달라고 요청

```
Pull Request
```

5



확인하고 합치기

```
리뷰 → Merge
```

6



내 main 맞추기

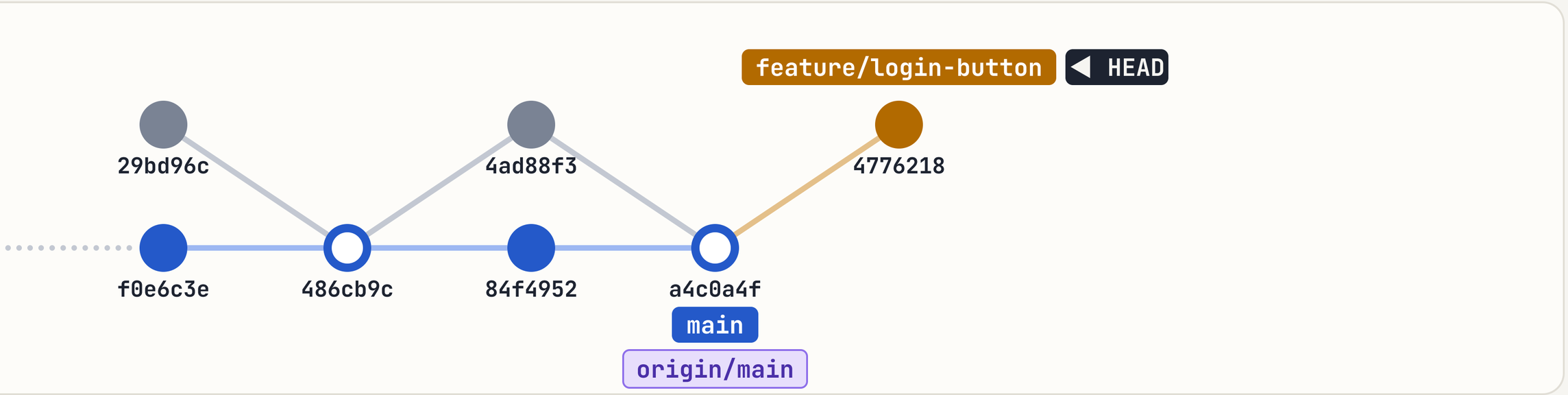
```
git checkout
main
git fetch
origin
git pull
origin main
```

main에는 직접 push하지 않아요. 6번을 마치면 다음 브랜치는 1번부터 다시 시작해요.

GitHub에서 함께

새 브랜치에서 로그인 버튼을 만들어요

```
$ git checkout -b feature/login-button
$ git add login.md
$ git commit -m "feat: 로그인 버튼 추가"
[feature/login-button 4776218] feat: 로그인 버튼 추가
```

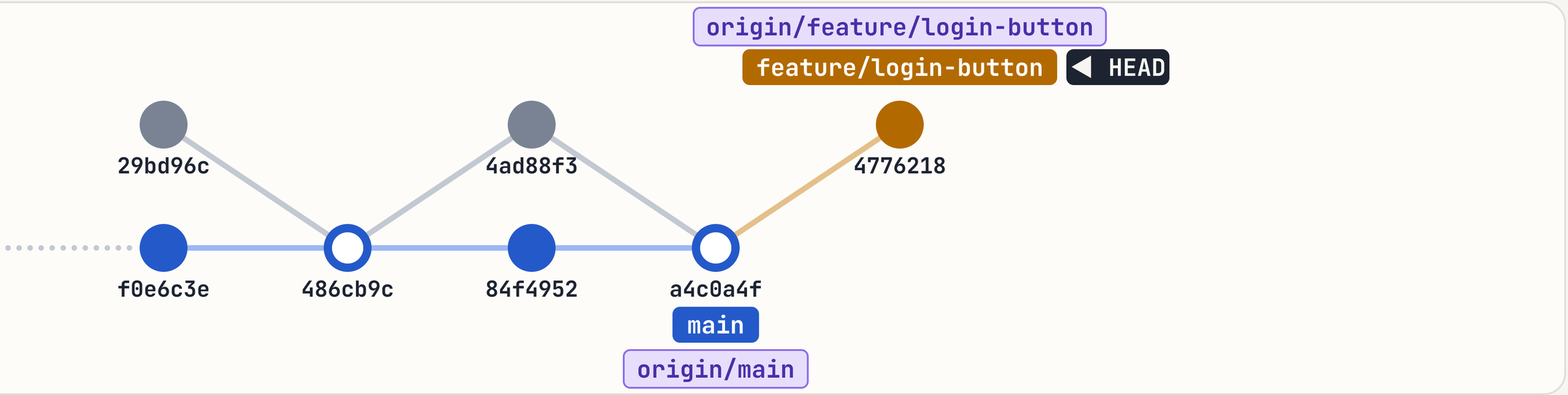


아직 내 컴퓨터에만 있는 커밋이에요.

GitHub에서 함께

브랜치째 GitHub에 올려요

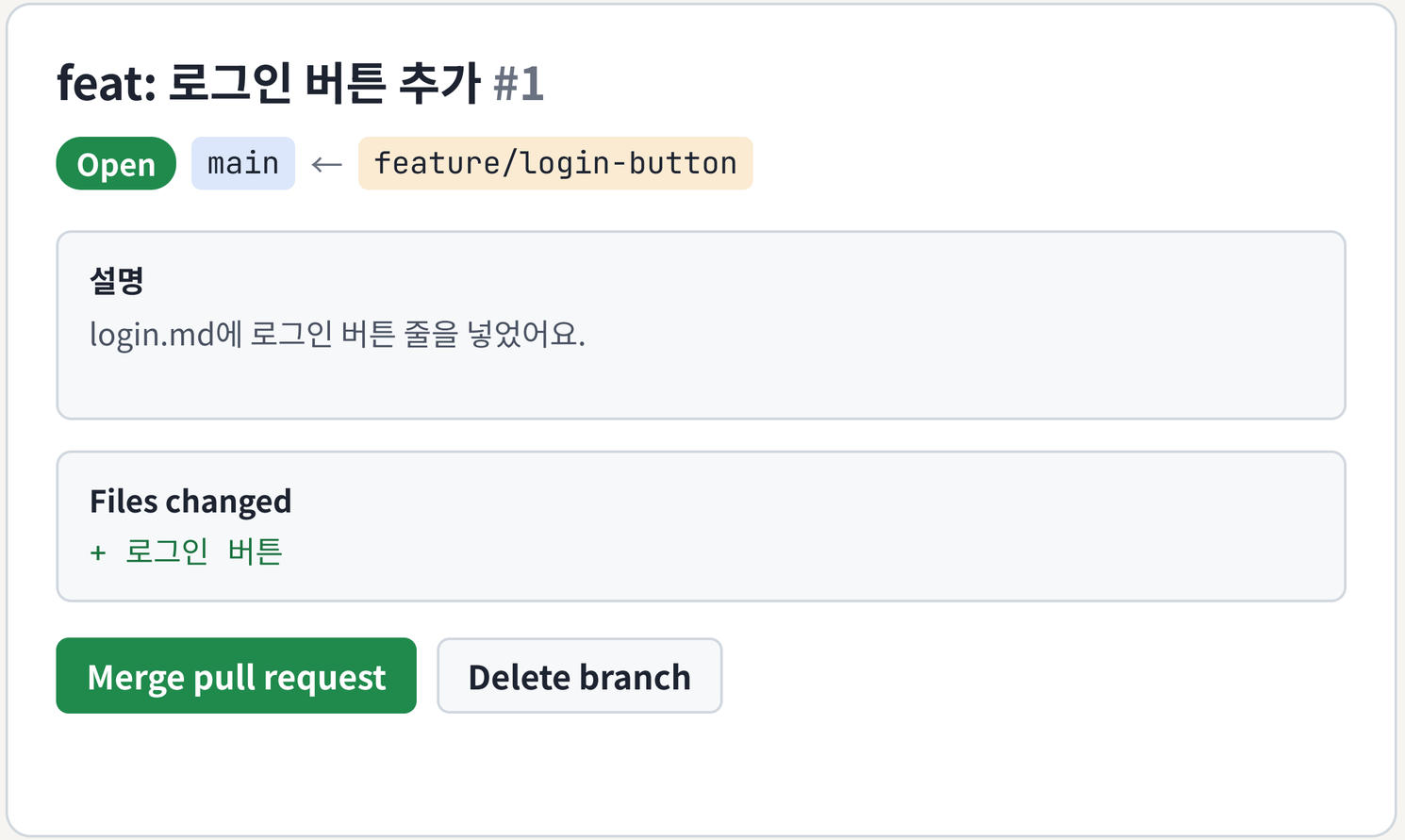
```
$ git push origin feature/login-button
 * [new branch] feature/login-button → feature/login-button
```



push할 때 main 대신 브랜치 이름을 끝까지 적어요.

GitHub에서 함께

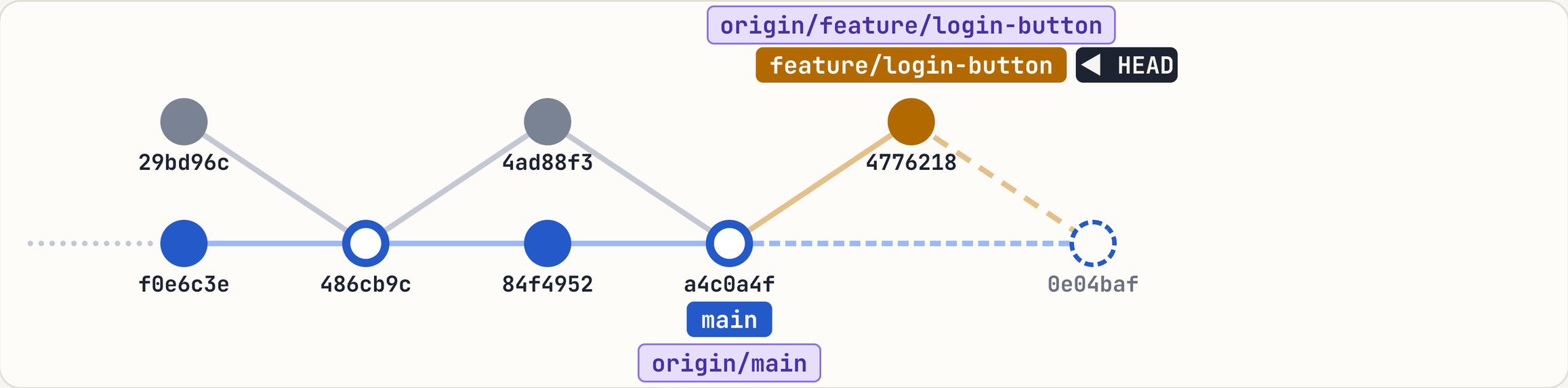
Pull Request 화면에서 할 일



1. 제목: 커밋 메시지처럼 타입: 요약
2. 어디로 ← 무엇을 합치는지
3. 무엇을 왜 바꿨는지
4. 바뀐 줄 확인, 댓글로 리뷰
5. 합치고 나면 GitHub 브랜치 삭제

GitHub에서 함께

GitHub에서 Merge하면 GitHub에만 커밋이 생겨요

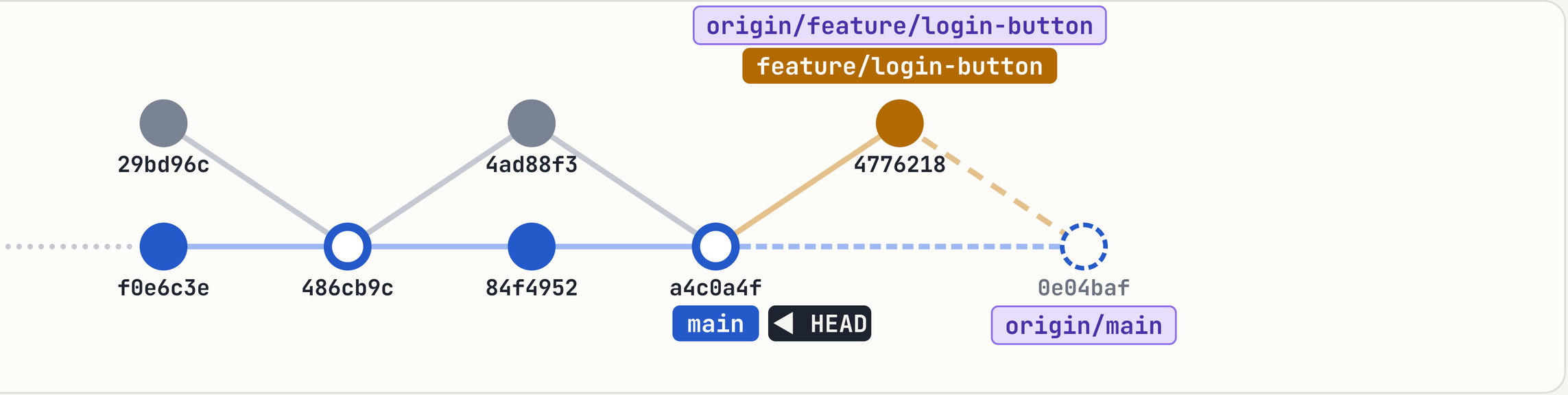


점선 커밋은 아직 GitHub에만 있어요. 내 컴퓨터는 몰라요.

GitHub에서 함께

main으로 가서 fetch로 받아만 와요

```
$ git checkout main
Switched to branch 'main'
$ git fetch origin
   a4c0a4f..0e04baf main → origin/main
```

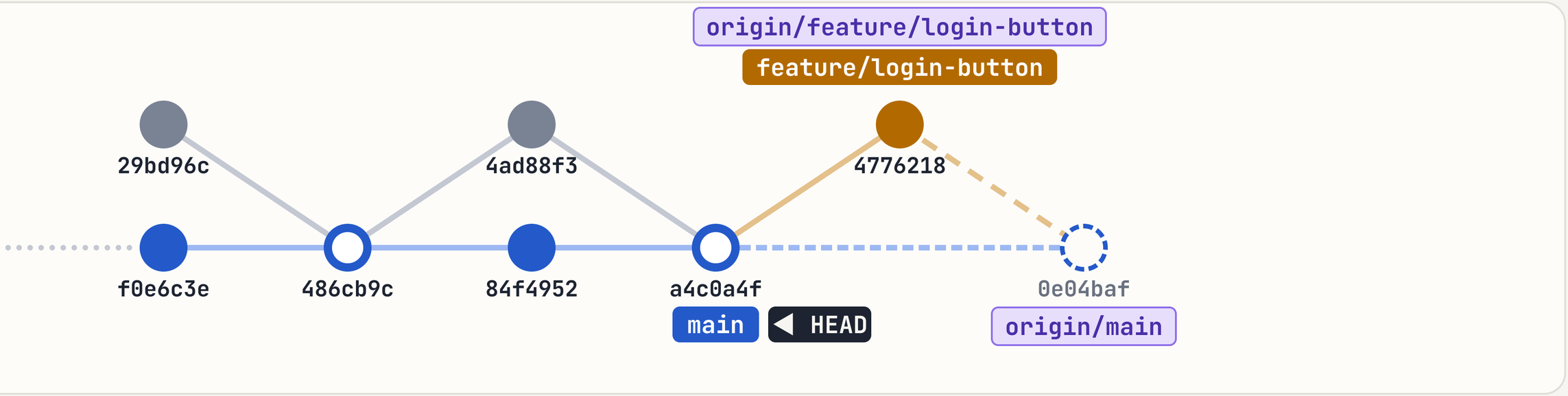


origin/main만 앞으로 갔어요. 내 main은 아직 그대로예요.

GitHub에서 함께

GitHub 쪽 기록을 먼저 확인해요

```
$ git log --oneline origin/main
0e04baf (origin/main) Merge pull request #1 from kim/feature/login-button
4776218 (origin/feature/login-button, feature/login-button) feat: 로그인 버튼 추가
a4c0a4f (HEAD → main) fix: 로그인 제목 충돌 해결
```

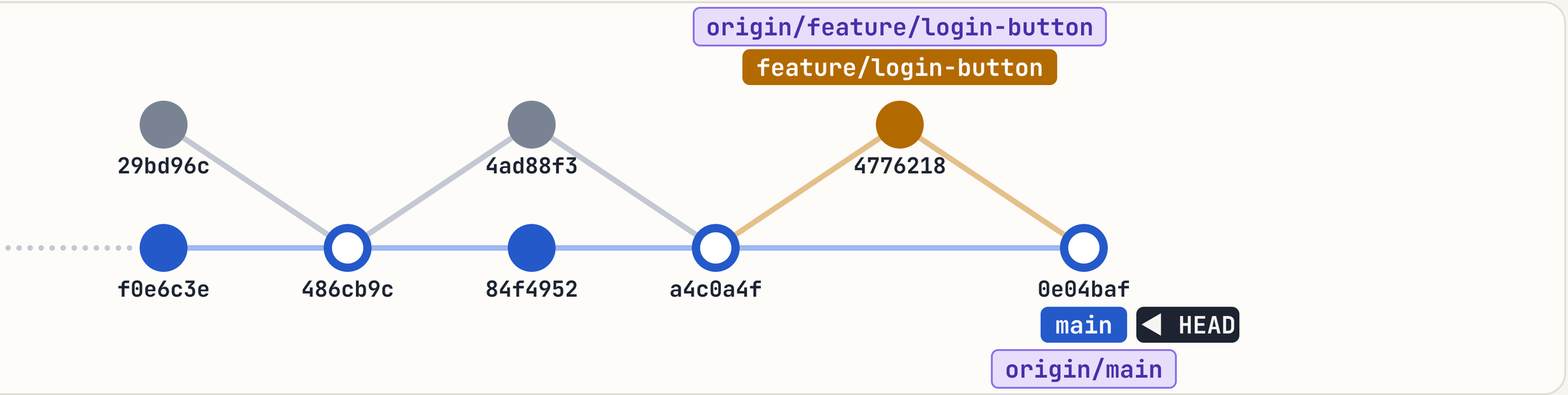


origin/main이 main보다 위에 있으면 pull할 차례예요.

GitHub에서 함께

pull로 내 main을 맞춰요

```
$ git pull origin main
Updating a4c0a4f..0e04baf
Fast-forward
```

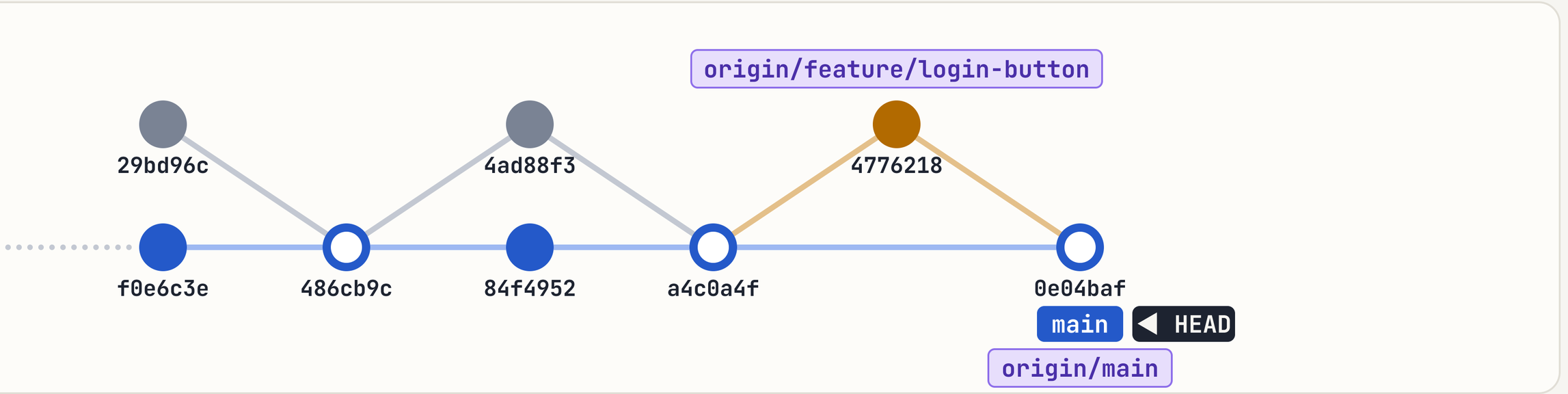


main이 origin/main과 같은 곳에 섰어요.

GitHub에서 함께

합쳐진 feature/login-button는 지워요

```
$ git branch -d feature/login-button
Deleted branch feature/login-button (was 4776218).
```

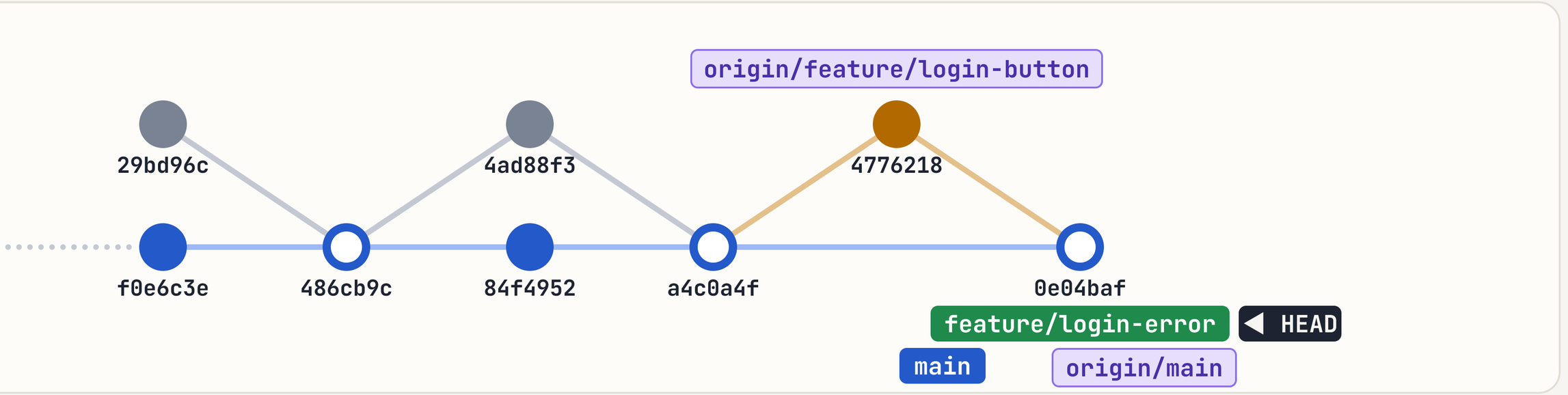


커밋은 main 기록 안에 남아요.

GitHub에서 함께

다음 일은 최신 main에서 새 브랜치로

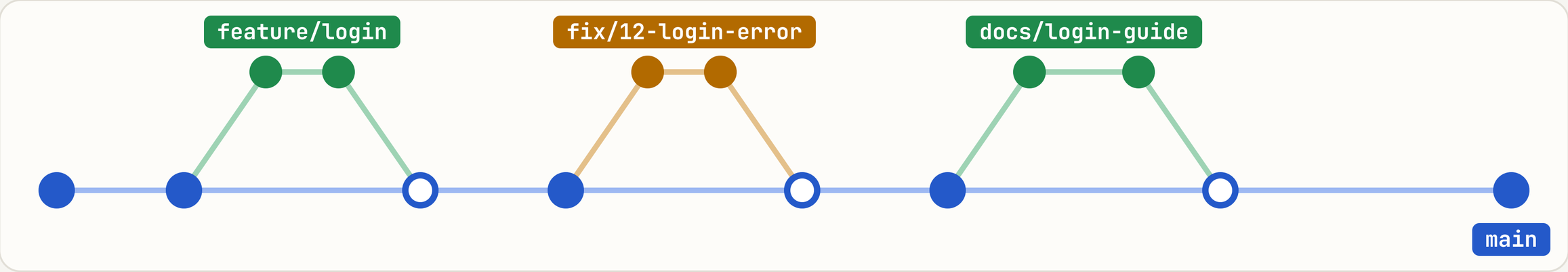
```
$ git checkout -b feature/login-error
Switched to a new branch 'feature/login-error'
```



합치고, main으로 가서 받고, 새로 따요. 브랜치마다 이 순서를 반복해요.

GitHub에서 함께 · 브랜치 전략

우리는 GitHub Flow로 일해요



main은 늘 동작하는 상태.
직접 커밋하지 않아요

일 하나 = 브랜치 하나.
이름은 타입/하는 일

작게 자주 Pull Request

merge 뒤 브랜치는 지워요

Git Flow(develop, release 브랜치를 따로 두는 방식)는 배포 일정이 복잡한 회사에서 써요. 처음엔 GitHub Flow면 충분해요.

충돌 시나리오 2 · 바로 옆 줄

다른 줄인데도 충돌이 나요

main login.md

아이디

비밀번호 (8자 이상)

feature/login-email login.md

이메일

비밀번호

```
$ git merge feature/login-email
CONFLICT (content): Merge conflict in login.md
Automatic merge failed; fix conflicts and then commit the result.
```

붙어 있는 줄을 각자 고치면 Git은 한 덩어리로 봐요.

충돌 시나리오 2 · 바로 옆 줄

둘 다 남기면 돼요

login.md (충돌 표시)

```
<<<<<<< HEAD
아이디
비밀번호 (8자 이상)
=======
이메일
비밀번호
>>>>>>> feature/login-email
```



main 고친 login.md

이메일

비밀번호 (8자 이상)

```
$ git add login.md
$ git commit -m "fix: 로그인 입력 충돌 해결"
[main a7c76b5] fix: 로그인 입력 충돌 해결
```

양쪽에서 바뀐 줄을 하나씩 가져와 합치고, 표시 세 줄은 지워요.

충돌 시나리오 3 · 지움 vs 고침

한쪽은 파일을 지우고, 한쪽은 고쳤어요

main old-login.md

임시 로그인 화면

아이디

chore/remove-old-login old-login.md

~~파일 삭제~~

```
$ git merge chore/remove-old-login
CONFLICT (modify/delete): old-login.md deleted in chore/remove-old-login and modified in HEAD.
Automatic merge failed; fix conflicts and then commit the result.
```

파일 안에 충돌 표시가 없어요. 지울지 남길지만 정하면 돼요.

충돌 시나리오 3 · 지움 vs 고침

지울지 남길지 골라요

```
$ git status
Unmerged paths:
(use "git add/rm <file>..." as appropriate to mark resolution)
    deleted by them: old-login.md
```

⚠ 지우기로 하면

```
git rm old-login.md
git commit -m "..."
```

✓ 남기기로 하면

```
git add old-login.md
git commit -m "..."
```

deleted by them은 합치려는 브랜치(them)가 지웠다는 뜻이에요.

충돌 시나리오 4 · 같은 이름 새 파일

둘 다 signup.md를 새로 만들었어요

main signup.md (새 파일)

```
- 전화번호
```

feature/signup signup.md (새 파일)

```
- 이메일
```

```
$ git merge feature/signup
CONFLICT (add/add): Merge conflict in signup.md
Automatic merge failed; fix conflicts and then commit the result.
```

처음부터 내용이 달라서, 파일 전체가 충돌 표시로 묶여요.

충돌 시나리오 4 · 같은 이름 새 파일

목록이면 둘 다 살려요

signup.md (충돌 표시)

```
<<<<<<< HEAD
- 전화번호
=======
- 이메일
>>>>>>> feature/signup
```



main 고친 signup.md

- 이메일
- 전화번호

```
$ git add signup.md
$ git commit -m "fix: 회원가입 항목 합치기"
[main 23d0c5f] fix: 회원가입 항목 합치기
```

푸는 방법은 시나리오 1과 같아요. 고르고, 표시 지우고, add, commit.

충돌 시나리오 5 · pull 할 때

팀원이 먼저 같은 줄을 push했어요

origin/main GitHub의 login.md (팀원)

```
# Login
```

아이디

main 내 login.md

```
# 로그인하기
```

아이디

```
$ git push origin main
! [rejected] main → main (fetch first)
$ git pull origin main
fatal: Need to specify how to reconcile divergent branches.
```

갈라진 상태에서 pull하면 어떻게 합칠지 정해 달라며 멈춰요.

충돌 시나리오 5 · pull 할 때

처음 한 번만 설정해요

```
git config --global pull.rebase false
```

git pull

pull은 두 단계예요

= fetch

GitHub 커밋을 받아 오고

+ merge

내 main과 합쳐요

pull.rebase false는 pull할 때 merge로 합치라는 뜻이에요. 설정한 뒤 다시 pull해요.

충돌 시나리오 5 · pull 할 때

다시 pull하면 익숙한 충돌이에요

```
$ git pull origin main
CONFLICT (content): Merge conflict in login.md
```

login.md (충돌 표시)

```
<<<<<<< HEAD
# 로그인하기
=======
# Login
>>>>>>> 82b872cd8ac7e79ce6c6eb09ee3531e9108dff77
```

```
$ git add login.md
$ git commit -m
"fix: 팀원과 로그인 제목 충돌 해결"
$ git push origin main
82b872c..1ebdd15 main → main
```

pull 충돌은 아래쪽 이름이 브랜치 대신 커밋 해시예요. 푸는 법은 같아요.

충돌 시나리오 6 · Pull Request

PR 화면에 충돌이 떠요

feat: 로그인 제목 바꾸기 #2

Open `main` ← `feature/login-title`

⚠ **This branch has conflicts that must be resolved**

```
Conflicting files: login.md
```

Merge pull request

내가 PR을 올린 사이 main이 같은 줄을 바꿨어요. Merge 버튼이 막혀요.

충돌 시나리오 6 · Pull Request

내 브랜치에서 main을 받아 풀고, 다시 push

```
$ git checkout feature/login-title
$ git pull origin main
CONFLICT (content): Merge conflict in login.md
# login.md 충돌 표시 지우고 저장
$ git add login.md
$ git commit -m "fix: main과 로그인 제목 충돌 해결"
$ git push origin feature/login-title
c7149cd..8feb440 feature/login-title → feature/login-title
```

push하면 PR 화면이 다시 Merge 할 수 있는 상태로 바뀌어요.

충돌 시나리오

메시지로 충돌 종류를 알아봐요

`CONFLICT (content)`	**같은 줄, 옆 줄**	고르고, 표시 지우고, add, commit
`CONFLICT (modify/delete)`	**지움 vs 고침**	git rm 또는 git add, commit
`CONFLICT (add/add)`	**같은 이름 새 파일**	고르고, 표시 지우고, add, commit
`fatal: Need to specify how to reconcile ...`	**갈라진 채 pull**	git config --global pull.rebase false
`This branch has conflicts ...`	**PR 충돌**	내 브랜치에서 pull origin main, 풀고 push

어떤 충돌이든 막히면 git merge --abort로 처음으로 돌아갈 수 있어요.

에러 1 · 브랜치 이름

옮기려는데, 만들려는데 안 돼요

없는 브랜치로 옮김

```
$ git checkout feature/nope
error: pathspec 'feature/nope' did not match any
file(s) known to git
```

이름 오타이거나 아직 안 만든 브랜치예요

```
git branch
```

```
git checkout -b feature/nope
```

이미 있는 이름으로 만듦

```
$ git checkout -b feature/login
fatal: a branch named 'feature/login' already
exists
```

같은 이름이 벌써 있어요. -b 없이 옮겨 가요

```
git checkout feature/login
```

에러 2 · 커밋 안 한 변경

브랜치를 옮기려는데 막혀요

고친 파일이 그대로 있음

```
error: Your local changes to the following files would be overwritten by checkout:
        login.md
Please commit your changes or stash them before you switch branches.
Aborting
```

옮겨 가면 고친 내용이 덮어써져서 Git이 멈췄어요. 먼저 커밋하고 옮겨요

```
git add login.md
```

```
git commit -m "feat: ..."
```

```
git checkout feature/login
```

에러 3 · 브랜치 지우기

브랜치가 안 지워져요

아직 합치지 않은 커밋이 있음

```
$ git branch -d feature/login
error: the branch 'feature/login' is not fully merged.
If you are sure you want to delete it, run 'git branch -D feature/login'
```

지우면 그 커밋을 잃을 수 있어서 Git이 멈췄어요

```
git checkout main
```

```
git merge feature/login
```

```
git branch -d feature/login
```

한 장 요약

브랜치

```
git branch
git branch 이름
git checkout 이름
git checkout -b 이름
git branch -d 이름

switch로 써도 돼요
git switch 이름
git switch -c 이름
```

합치기

```
git checkout main
git merge 이름

push 전에 취소
git reset --hard HEAD~1
```

충돌

```
파일에서 고르기
표시 세 줄 지우기
git add 파일
git commit -m "..."
막히면 git merge --abort
```

GitHub과 함께

```
git push origin 이름
→ Pull Request
→ Merge
git checkout main
git fetch origin
git pull origin main
```

브랜치 이름: feature/login-button · main에는 직접 커밋하지 않기